

ArtisanHub — Guide complet de déploiement AWS EC2

Ubuntu • Laravel • MySQL • Nginx • PHP-FPM • Supervisor • HTTPS

Ce guide concerne une instance **Ubuntu EC2 déjà créée**. Il privilégie une architecture peu coûteuse pour un pilote ou une démonstration :

```
Une instance EC2
├── Laravel ArtisanHub
├── PHP 8.3 + PHP-FPM
├── Nginx
├── MySQL local
├── Supervisor pour la queue
└── Cron pour le scheduler Laravel
```

Cette configuration n'est pas une haute disponibilité et ne remplace pas une infrastructure de production complète.

1. Vérifier AWS avant de commencer

Dans **EC2**, vérifier :

- instance Ubuntu démarrée ;
- type d'instance inclus dans ton offre gratuite ou couvert par tes crédits ;
- disque EBS raisonnable, par exemple 8 à 20 Go ;
- absence de NAT Gateway ;
- absence de Load Balancer ;
- absence de RDS au début ;
- région AWS correcte ;
- adresse IPv4 publique disponible.

Security Group

Autoriser uniquement :

Type	Port	Source
SSH	22	My IP uniquement
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0

Ne pas ouvrir publiquement les ports 3306, 5432 ou 6379.

2. Se connecter à Ubuntu

Depuis ton ordinateur :

```
chmod 400 ~/Téléchargements/artisanhub.pem  
ssh -i ~/Téléchargements/artisanhub.pem ubuntu@ADRESSE_IP_EC2
```

Adapter le chemin de la clé et l'adresse IP.

Vérifier le serveur :

```
whoami  
lsb_release -a  
free -h  
df -h
```

3. Ajouter un swap pour les petites instances

Cette étape évite les erreurs de mémoire pendant Composer ou npm :

```
sudo fallocate -l 2G /swapfile  
sudo chmod 600 /swapfile  
sudo mkswap /swapfile  
sudo swapon /swapfile  
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab  
free -h
```

4. Installer les paquets système

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install -y \  
  git curl unzip zip wget ca-certificates \  
  software-properties-common ufw build-essential
```

5. Installer PHP 8.3

Laravel 12 demande PHP 8.2 minimum. PHP 8.3 est recommandé :

```
sudo add-apt-repository ppa:ondrej/php -y  
sudo apt update  
sudo apt install -y \  
  php8.3 php8.3-fpm php8.3-cli php8.3-mysql \  
  php8.3-xml php8.3-curl php8.3-mbstring \  
  php8.3-zip php8.3-gd php8.3-intl \  
  php8.3-bcmath php8.3-opcache  
php -v  
sudo systemctl enable --now php8.3-fpm
```

6. Installer Nginx

```
sudo apt install -y nginx  
sudo systemctl enable --now nginx
```

Tester dans le navigateur :

```
http://ADRESSE_IP_EC2
```

7. Installer et sécuriser MySQL

```
sudo apt install -y mysql-server
sudo systemctl enable --now mysql
sudo mysql_secure_installation
```

Créer la base et l'utilisateur :

```
sudo mysql

CREATE DATABASE artisanhub
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'artisanhub_user'@'localhost'
  IDENTIFIED BY 'REPLACER_PAR_UN_MOT_DE_PASSE_FORT';

GRANT ALL PRIVILEGES ON artisanhub.*
  TO 'artisanhub_user'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

Ne jamais utiliser le compte root MySQL dans `.env`.

8. Installer Composer et Node.js

Composer

```
cd /tmp
curl -sS https://getcomposer.org/installer -o composer-setup.php
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
rm composer-setup.php
composer --version
```

Node.js

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

9. Envoyer le projet sur GitHub

Depuis ton ordinateur local :

```
cd /home/landry/artisanhub.worktrees/greeting-in-french
git remote add origin https://github.com/TON_COMPTE/TON_REPO.git
git push -u origin main
```

Si le remote existe déjà :

```
git remote -v
git push origin main
```

Ne jamais pousser `.env`, clés API ou mots de passe.

10. Cloner ArtisanHub sur EC2

```
sudo mkdir -p /var/www
sudo chown -R ubuntu:ubuntu /var/www
cd /var/www
git clone https://github.com/TON_COMPTE/TON_REPO.git artisanhub
cd /var/www/artisanhub
```

Pour un dépôt privé, utiliser une clé SSH GitHub ou un token limité. Ne pas publier le token dans un script.

11. Installer les dépendances

```
cd /var/www/artisanhub
composer install --no-dev --optimize-autoloader --no-interaction
npm ci
npm run build
```

12. Configurer .env

```
cp .env.example .env
nano .env
```

Configuration économique initiale :

```
APP_NAME=ArtisanHub
APP_ENV=production
APP_DEBUG=false
APP_URL=http://ADRESSE_IP_EC2

APP_LOCALE=fr
APP_FALLBACK_LOCALE=fr

LOG_CHANNEL=single
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=artisanhub
DB_USERNAME=artisanhub_user
DB_PASSWORD=MOT_DE_PASSE_MYSQL

SESSION_DRIVER=database
SESSION_LIFETIME=120
CACHE_STORE=database
QUEUE_CONNECTION=database
FILESYSTEM_DISK=local

MAIL_MAILER=log
MAIL_FROM_ADDRESS=no-reply@example.com
MAIL_FROM_NAME=ArtisanHub

FEDAPAY_ENV=sandbox
WHATSAPP_DRIVER=log
```

Générer la clé :

```
php artisan key:generate --force
```

Ne jamais régénérer APP_KEY après le lancement sans procédure de migration des données chiffrées.

13. Initialiser Laravel

```
php artisan migrate --force
php artisan storage:link
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Créer la table des jobs si elle n'existe pas :

```
php artisan queue:table
php artisan migrate --force
```

Ne jamais exécuter `migrate:fresh` en production.

14. Configurer les permissions

```
sudo chown -R www-data:www-data /var/www/artisanhub
sudo find /var/www/artisanhub -type d -exec chmod 755 {} \;
sudo find /var/www/artisanhub -type f -exec chmod 644 {} \;
sudo chmod -R 775 /var/www/artisanhub/storage
sudo chmod -R 775 /var/www/artisanhub/bootstrap/cache
sudo chmod 640 /var/www/artisanhub/.env
```

15. Configurer Nginx

```
sudo nano /etc/nginx/sites-available/artisanhub

server {
    listen 80;
    listen [::]:80;
    server_name ADRESSE_IP_EC2;

    root /var/www/artisanhub/public;
    index index.php index.html;
    charset utf-8;
    client_max_body_size 10M;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    location ~ /\.(!well-known).* {
        deny all;
    }
}
```

```
    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;
}
```

Activer :

```
sudo ln -s /etc/nginx/sites-available/artisanhub
/etc/nginx/sites-enabled/artisanhub
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

16. Configurer UFW

Remplacer TON_ADRESSE_IP par ton IP publique actuelle :

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from TON_ADRESSE_IP to any port 22 proto tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status verbose
```

Si ton IP change et que SSH est bloqué, modifier la règle depuis la console AWS.

17. Configurer le worker Laravel

```
sudo apt install -y supervisor
sudo systemctl enable --now supervisor
sudo nano /etc/supervisor/conf.d/artisanhub-worker.conf

[program:artisanhub-worker]
process_name=%(program_name)s
command=php /var/www/artisanhub/artisan queue:work database --sleep=3 --tries=3
--timeout=90 --max-time=3600
directory=/var/www/artisanhub
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=1
redirect_stderr=true
stdout_logfile=/var/log/artisanhub-worker.log
stopwaitsecs=3600

sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl status
```

Le résultat attendu est RUNNING.

18. Configurer le scheduler Laravel

```
sudo crontab -u www-data -e
```

Ajouter :

```
* * * * * cd /var/www/artisanhub && php artisan schedule:run >> /dev/null 2>&1
```

19. Activer HTTPS avec un domaine

Un domaine est nécessaire pour un HTTPS propre, les webhooks et la PWA.

Dans le DNS du domaine :

A	@	ADRESSE_IP_EC2
A	www	ADRESSE_IP_EC2

Vérifier :

```
dig +short ton-domaine.com
```

Installer Certbot :

```
sudo apt install -y certbot python3-certbot-nginx
```

Changer server_name dans Nginx :

```
server_name ton-domaine.com www.ton-domaine.com;
```

Puis :

```
sudo nginx -t  
sudo systemctl reload nginx  
sudo certbot --nginx -d ton-domaine.com -d www.ton-domaine.com
```

Mettre à jour .env :

```
APP_URL=https://ton-domaine.com  
  
cd /var/www/artisanhub  
php artisan optimize:clear  
php artisan config:cache
```

20. Configurer FedaPay

Commencer en Sandbox :

```
FEDAPAY_ENV=sandbox  
FEDAPAY_PUBLIC_KEY=cle_sandbox  
FEDAPAY_SECRET_KEY=cle_sandbox  
FEDAPAY_WEBHOOK_SECRET=secret_webhook
```

Lister les routes de paiement :

```
php artisan route:list | grep -i paiement
```

Utiliser dans FedaPay une URL HTTPS correspondant réellement à la route du projet, par exemple :

```
https://ton-domaine.com/paiement/callback
```

Tester paiement, callback, confirmation, remboursement et reversement avant de passer en production.

21. Configurer les emails

Pour le premier déploiement :

```
MAIL_MAILER=log
```

Pour un SMTP réel :

```
MAIL_MAILER=smtp
MAIL_HOST=smtp-relay.brevo.com
MAIL_PORT=587
MAIL_USERNAME=ton-email
MAIL_PASSWORD=cle-smtp
MAIL_FROM_ADDRESS=no-reply@ton-domaine.com
MAIL_FROM_NAME=ArtisanHub
```

Après modification :

```
cd /var/www/artisanhub
php artisan optimize:clear
php artisan config:cache
sudo supervisorctl restart artisanhub-worker
```

22. Stockage des fichiers

Pour une démonstration :

```
FILESYSTEM_DISK=local
```

Pour un pilote plus sérieux, configurer S3 ou R2 :

```
FILESYSTEM_DISK=s3
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_DEFAULT_REGION=...
AWS_BUCKET=artisanhub
AWS_ENDPOINT=
AWS_URL=
AWS_USE_PATH_STYLE_ENDPOINT=false
```

Tester l'upload d'un document KYC et d'une preuve de livraison. Ne pas utiliser une clé AWS root.

23. Vérifications après déploiement

```
curl -i http://ADRESSE_IP_EC2/up
php artisan about
php artisan migrate:status
sudo systemctl status nginx
sudo systemctl status php8.3-fpm
sudo supervisorctl status
```

Avec HTTPS :

```
curl -i https://ton-domaine.com/up
curl -i https://ton-domaine.com/manifest.webmanifest
curl -i https://ton-domaine.com/sw.js
```

Tester ensuite inscription, commande, paiement Sandbox, support, upload, queue et dashboards.

24. Commandes de maintenance

```
# Logs Laravel
tail -f /var/www/artisanhub/storage/logs/laravel.log

# Logs Nginx
sudo tail -f /var/log/nginx/error.log

# Worker
sudo supervisorctl status
sudo supervisorctl restart artisanhub-worker
tail -f /var/log/artisanhub-worker.log

# Jobs échoués
cd /var/www/artisanhub
php artisan queue:failed
php artisan queue:retry all

# Cache
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

25. Mise à jour du projet

```
cd /var/www/artisanhub
php artisan down
git pull origin main
composer install --no-dev --optimize-autoloader --no-interaction
npm ci
npm run build
php artisan migrate --force
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
sudo supervisorctl restart artisanhub-worker
sudo systemctl reload php8.3-fpm
sudo systemctl reload nginx
php artisan up
```

26. Sauvegarde MySQL

Créer le dossier :

```
sudo mkdir -p /var/backups/artisanhub
sudo chmod 700 /var/backups/artisanhub
```

Créer un fichier de credentials protégé :

```
sudo nano /root/.my.cnf

[client]
user=artisanhub_user
password=MOT_DE_PASSE_MYSQL

sudo chmod 600 /root/.my.cnf
```

Tester une sauvegarde :

```
sudo mysqldump artisanhub | gzip | sudo tee \  
/var/backups/artisanhub/artisanhub-$(date +%F-%H%M%S).sql.gz > /dev/null  
sudo ls -lh /var/backups/artisanhub
```

Ajouter une sauvegarde quotidienne :

```
sudo crontab -e  
  
0 2 * * * mysqldump artisanhub | gzip > /var/backups/artisanhub/artisanhub-$(  
date +%F-%H%M%S).sql.gz
```

Une sauvegarde sur la même instance ne protège pas contre la perte de l'instance. Copier périodiquement les archives vers un stockage externe.

27. Contrôler les coûts AWS

Créer un budget dans **Billing** → **Budgets** avec alertes à 50 %, 80 % et 100 %.

Surveiller :

- EC2 ;
- EBS ;
- Elastic IP ;
- S3 ;
- CloudWatch ;
- Route 53 ;
- NAT Gateway ;
- RDS.

Éviter au début :

- NAT Gateway ;
- Load Balancer ;
- RDS ;
- plusieurs instances ;
- Elastic IP non utilisée ;
- snapshots inutiles ;
- logs CloudWatch excessifs.

28. Ne pas lancer aveuglément deploy/DEPLOY.sh

Le projet contient un script d'installation, mais vérifie-le avant utilisation :

```
sed -n '1,240p' /var/www/artisanhub/deploy/DEPLOY.sh
```

Il installe notamment PHP, Nginx, MySQL, Node.js, Supervisor et UFW. La procédure manuelle de

ce guide est préférable lorsqu'une instance est déjà partiellement configurée.

29. Checklist finale

- ☐ Security Group configuré.
- ☐ Swap activé.
- ☐ PHP 8.3 installé.
- ☐ Nginx installé.
- ☐ MySQL installé.
- ☐ Base ArtisanHub créée.
- ☐ Projet cloné depuis GitHub.
- ☐ .env rempli et non publié.
- ☐ APP_KEY générée.
- ☐ Migrations exécutées.
- ☐ Permissions configurées.
- ☐ Nginx pointe vers /public.
- ☐ Worker Supervisor actif.
- ☐ Cron Laravel actif.
- ☐ HTTPS activé si domaine disponible.
- ☐ Fedapay Sandbox testé.
- ☐ PWA testée en HTTPS.
- ☐ Sauvegarde MySQL testée.
- ☐ Budget AWS configuré.

30. Références

- [AWS Free Tier](#)
- [AWS EC2](#)
- [AWS Free Tier FAQ](#)
- [Laravel Deployment](#)
- [Let's Encrypt](#)
- [Fedapay Documentation](#)